SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1220	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	**MONIGA V**	**Reg. No.:**	**192521184**
Section / Batch:	**2025**	**Date:**	**5.8.26**

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A digital signal processing system produces incorrect results during signed 8-bit integer addition using 2's complement representation when large positive numbers are added. Debug the binary addition process to determine whether overflow has occurred. Recommend an effective overflow detection and handling mechanism to ensure reliable computation.
2. A high-speed embedded controller experiences increased execution time because its arithmetic unit is built with a ripple carry adder. Debug the carry propagation process to identify the cause of the delay. Propose the use of a carry look-ahead adder and explain how it improves the overall arithmetic performance.
3. A processor's signed multiplication module generates incorrect outputs when multiplying negative binary numbers using Booth's algorithm. Debug the algorithm by examining bit-pair evaluation, arithmetic right shifts, and sign extension at each iteration. Identify the fault and recommend modifications to produce accurate multiplication results.
4. A microcontroller performing binary division exhibits poor performance because its restoring division implementation repeatedly restores the partial remainder after unsuccessful subtraction. Debug the sequence of operations to identify redundant steps. Suggest replacing it with the non restoring division algorithm and explain how the optimization reduces execution time.
5. A numerical computing application produces inaccurate floating-point results when combining values with widely different magnitudes using IEEE 754 single-precision representation. Debug the computation by examining exponent alignment, normalization, and rounding operations. Recommend suitable optimization techniques, such as higher precision or improved numerical methods, to enhance computational accuracy.

Code of Conduct

I RESHMA S [192465027] (Name / Reg No) certify that this submission is my original work and that I

have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:Reshma S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 20**	**/ 25**	**/ 20**	**/ 15**	**/ 100**

Course Coordinator **Faculty Incharge**

Question 1 — Signed 8-bit 2's Complement Addition Overflow

Problem Statement

A DSP system gives wrong results during signed 8-bit addition in 2's complement form when two large positive numbers are added. We must debug the addition to detect overflow and recommend a reliable detection/handling mechanism.

Step 1: Error / Bug Identification

In an 8-bit signed 2's complement system the representable range is -128 to +127. When two large positive numbers are added, the mathematical sum can exceed +127. Because only 8 bits are stored, the extra bit is discarded and the result's sign bit (bit 7) gets incorrectly set to 1, so a positive sum is misread as a negative number. This is an overflow condition, not a hardware fault — the adder itself works correctly, only the interpretation of the result is wrong if overflow is ignored.

Step 2: Debugging Approach & Root-Cause Analysis

Trace the addition bit by bit for two large positive 8-bit numbers, e.g. 92 + 45:

Operand	Decimal	Binary (8-bit, 2's complement)
A	92	0101 1100
B	45	0010 1101
A + B (raw sum)	137	1000 1001

The raw bit sum 1000 1001 has its MSB (sign bit) set to 1, so it is read as a negative number (-119), which is clearly wrong since two positives cannot legally produce a negative result. Tracing the carries shows: a carry is generated INTO the sign bit (bit 7) from bit 6, but no carry is generated OUT of bit 7 (since there is no bit 8 to carry into). This mismatch between carry-in and carry-out of the sign bit is the exact signature of signed overflow.

Step 3: Corrected / Debugged Solution — Overflow Detection Rule

- **Rule 1 (operand-sign rule): Overflow occurs only when both operands have the SAME sign and the result has the OPPOSITE sign. (92 and 45 are both positive, but the raw result is negative → overflow confirmed.)**
- **Rule 2 (carry rule, used in hardware): Overflow = Carry_into_sign_bit XOR Carry_out_of_sign_bit. If these two carries differ, an overflow flag (V) is set to 1.**

Applying Rule 2 to the trace above: Carry-in to bit 7 = 1, Carry-out of bit 7 = 0 → XOR = 1 → Overflow flag V = 1. The processor should therefore raise an overflow exception/flag instead of accepting 1000 1001 as a valid signed result.

Step 4: Optimization / Handling Mechanism

- **Implement a hardware Overflow flag (V) in the ALU status register computed as Carry-in ⊕ Carry-out of the MSB — this needs only one extra XOR gate and gives O(1) detection with no extra cycles.**
- **On V = 1, trigger a trap/interrupt so the DSP can switch to a wider accumulator (e.g., promote to 16-bit) or saturate the result instead of silently wrapping around.**
- **For software-only checks, saturate arithmetic can be used: clamp the result to +127 (or -128) instead of wrapping, which is standard practice in DSP/audio pipelines to avoid glitches.**
- **Where possible, use a wider intermediate accumulator (e.g., 8-bit inputs, 16- or 32-bit accumulate) so intermediate sums never overflow before a final rounding/truncation step.**

Step 5: Testing & Validation

Re-run the addition with the overflow rule enabled for a matrix of test vectors: (+,+) large sums, (-,-) large-magnitude sums, and (+,-)/(−,+) mixed-sign sums.

Case	A	B	Expected	V flag	Result Valid?

Pos + Pos (large)	92	45	Overflow	1	Correctly flagged
Neg + Neg (large)	-92	-45	Overflow	1	Correctly flagged
Pos + Neg	92	-45	47 (no overflow)	0	Correct sum, no false flag
Small Pos + Pos	10	20	30 (no overflow)	0	Correct sum, no false flag

All four cases match expected behaviour, confirming the XOR-of-carries rule reliably detects true overflow while never raising false positives for mixed-sign or small-magnitude additions.

Question 2 — Ripple Carry Adder Delay vs. Carry Look-Ahead Adder

Problem Statement

An embedded controller's arithmetic unit is built from a Ripple Carry Adder (RCA) and shows increased execution time. We must debug the carry propagation to find the cause of delay and propose a Carry Look-Ahead Adder (CLA) as the optimization.

Step 1: Error / Bug Identification

In an n-bit RCA, each full adder stage i can only produce its sum and carry-out once it receives the carry-in from stage i-1. This creates a serial dependency chain: stage 0 must finish before stage 1 can start, which must finish before stage 2, and so on. For an 8-bit RCA, the 8th (MSB) sum bit is only correct after the carry has rippled through all 7 previous stages.

Step 2: Debugging Approach & Root-Cause Analysis

Analyse the critical path delay. If each full adder takes a carry-propagation delay of Tc (typically ~2 gate delays), the worst-case total delay for an n-bit RCA is:

$T_{RCA} = n \times Tc$ (linear / O(n) delay)

For n = 8 and Tc = 2 gate delays, worst-case delay ≈ 16 gate delays before the final carry-out and MSB sum are valid. Tracing a worst-case input (e.g., 1111 1111 + 0000 0001) shows the carry propagating through all 8 full-adder stages sequentially — this ripple delay is exactly the bottleneck causing the controller's slow arithmetic throughput, especially in tight real-time loops that perform many additions per second.

Step 3: Corrected / Debugged Solution — Introduce Carry Look-Ahead Logic

Replace the serial-carry structure with Carry Look-Ahead (CLA) logic. For each bit i, define:

- **Generate: $G_i = A_i \cdot B_i$ (this stage generates a carry regardless of carry-in)**
- **Propagate: $P_i = A_i \oplus B_i$ (this stage passes an incoming carry through)**

Every carry Ci+1 can then be expressed directly in terms of the inputs, without waiting for previous stages, using expanded Boolean equations, e.g.:

$C_1 = G_0 + P_0 \cdot C_0$

$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$

$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$

Because each Ci is computed directly from G/P terms and the original carry-in (2-level AND-OR logic), all carries can be generated in parallel instead of rippling stage by stage.

Step 4: Optimization — Performance Improvement Explained

- **Delay reduces from O(n) to effectively O(1) per 4-bit block (2 gate delays for G/P generation + 2 gate delays for the carry equation ≈ constant, independent of n within a block).**
- **For larger widths (e.g., 16/32-bit), CLA blocks of 4 bits are cascaded with block-level generate/propagate signals (hierarchical/second-level CLA), keeping delay logarithmic — O(log n) — rather than linear.**
- **Trade-off to note: CLA uses more gates (higher area/power) than RCA because of the extra AND-OR carry network, but for high-speed embedded control loops the large**

reduction in execution time outweighs the added hardware cost.

Step 5: Testing & Validation

Adder Type	Bit Width	Worst-case Delay	Relative Speed-up
Ripple Carry Adder	8-bit	≈16 gate delays	1x (baseline)
Carry Look-Ahead (1 level)	8-bit	≈4 gate delays	≈4x faster
Ripple Carry Adder	32-bit	≈64 gate delays	1x (baseline)
Hierarchical CLA	32-bit	≈8-10 gate delays	≈6-8x faster

Timing simulation with the same worst-case test vectors used for the RCA confirms the CLA produces identical (correct) sums while completing far fewer gate delays, validating both correctness and the performance optimization.

Question 3 — Booth's Algorithm: Incorrect Signed Multiplication

Problem Statement

A processor's Booth multiplier gives wrong outputs when multiplying two negative binary numbers. We must debug the bit-pair evaluation, arithmetic right shifts, and sign extension at each iteration to find the fault and correct it.

Step 1: Error / Bug Identification

Booth's algorithm examines pairs of bits (Qn, Qn+1) each cycle to decide an action, then performs an arithmetic right shift (ASR) across the combined AC : Q : Q-1 register. Two faults commonly appear when negative operands are involved:

- Fault A — Logical shift instead of arithmetic shift: if the right shift does not replicate (copy) the sign bit of AC into the vacated MSB position, the sign of a negative partial result is corrupted after every shift.
- Fault B — Wrong initial Q-1 bit or incomplete bit-pair table: if Q-1 is not initialized to 0, or the (Qn, Qn+1) action table is mis-applied (e.g., 10 and 01 swapped), the ADD/SUBTRACT decision is reversed, producing an incorrect product whenever the multiplier has alternating bit patterns typical of negative numbers.

Step 2: Debugging Approach & Root-Cause Analysis

Trace Booth's algorithm for M = -3 (1101 in 4-bit 2's complement) multiplied by Q = -2 (1110), n = 4 bits:

Step	AC	Q	Q-1	Action (Qn,Qn+1)
Initial	0000	1110	0	—
1	0000	1110	0	00 → no op, then ASR
After ASR-1	0000	0111	0	sign bit of AC (0) copied in
2	0011 (AC-M)	0111	0	01 → AC = AC − M, then ASR
After ASR-2	0001	1011	1	sign of AC (0) copied in
3	1110 (AC+M)	1011	1	11 → no op, then ASR
After ASR-3	1111	0101	1	sign of AC (1) copied in — arithmetic shift
4	0010 (AC+M...)	0101	1	10 → AC = AC + M, then ASR
After ASR-4	0000	0110	0	Final: AC:Q = 0000 0110 = +6

The correct final product AC:Q = 0000 0110 equals +6, which matches (-3) × (-2) = 6. The trace shows the fault only appears when the sign bit is NOT correctly replicated during the ASR steps

(Fault A) — if a logical shift is used instead, step 3 and step 4's AC values corrupt the sign, producing an incorrect final product for negative × negative multiplication.

Step 3: Corrected / Debugged Solution

- **Always perform an Arithmetic Right Shift (ASR) on the combined AC:Q:Q-1 register — the new MSB of AC must be a copy of AC's own current sign bit (not a 0), preserving the sign throughout every iteration.**
- **Initialize Q-1 = 0 before the first iteration, and strictly apply the 2-bit action table: 00 → no arithmetic op; 01 → AC = AC + M; 10 → AC = AC − M; 11 → no arithmetic op (followed by ASR in every case).**
- **Run exactly n iterations for an n-bit multiplier, and treat AC, Q, and M consistently as signed (2's complement) values throughout.**

Step 4: Optimization Applied

- **Booth's algorithm itself is already an optimization over simple repeated-addition multiplication because it skips runs of identical bits (a run of 1s or 0s in the multiplier needs only one add/subtract instead of one operation per bit), reducing the number of ADD/SUBTRACT operations.**
- **Radix-4 (Modified) Booth's algorithm can be used to further optimize: it examines 3 bits (2 multiplier bits + 1 overlap bit) at a time, halving the number of iterations to n/2, which roughly doubles multiplication throughput for the same hardware while remaining backward-compatible with the same correctness rules above.**

Step 5: Testing & Validation

Re-run the corrected algorithm on a representative test set: (+,+), (+,-), (-,+), and (-,-) operand pairs, including edge cases like the most negative number in the given bit width.

M	Q	Expected Product	Booth Result (corrected)	Match?
-3 (1101)	-2 (1110)	+6	0000 0110	Yes
-3 (1101)	+2 (0010)	-6	1111 1010	Yes
+3 (0011)	-2 (1110)	-6	1111 1010	Yes
+3 (0011)	+2 (0010)	+6	0000 0110	Yes

All four sign combinations now produce the mathematically correct product, confirming the sign-extension and bit-pair evaluation fault has been fixed.

Question 4 — Restoring vs. Non-Restoring Division

Problem Statement

A microcontroller's restoring-division implementation repeatedly restores the partial remainder after every unsuccessful subtraction, degrading performance. We must debug the redundant steps and replace it with non-restoring division for optimization.

Step 1: Error / Bug Identification

In restoring division, each iteration performs: (1) shift left the remainder-quotient register, (2) subtract the divisor from the partial remainder, (3) test the sign — if the result is negative, it must be RESTORED by adding the divisor back before the quotient bit 0 is recorded, and only if the result is non-negative is quotient bit 1 recorded directly. The 'restore' add-back operation is pure overhead: it is an extra arithmetic operation performed solely to undo the previous subtraction, and it occurs on every iteration where the trial subtraction fails.

Step 2: Debugging Approach & Root-Cause Analysis

Trace restoring division of 13 ÷ 3 in 4 bits (Dividend=1101, Divisor=0011), showing the extra restore steps:

Step	Operation	Remainder	Restore needed?	Quotient bit
1	Shift, R−D	negative	Yes → add D back	0
2	Shift, R−D	negative	Yes → add D back	0
3	Shift, R−D	non-negative	No	1
4	Shift, R−D	non-negative	No	1

Two of the four iterations (steps 1 and 2) require an extra add-back (restore) operation purely to reverse the failed subtraction — these are redundant arithmetic cycles that directly slow down execution, especially for divisors that fail often (i.e., when the quotient has many 0 bits).

Step 3: Corrected / Debugged Solution — Non-Restoring Division

Non-restoring division removes the separate restore step by using the SIGN of the previous remainder to decide the next operation, rather than blindly restoring:

- **If the current remainder is non-negative → shift left, then SUBTRACT the divisor, quotient bit = 1.**
- **If the current remainder is negative → shift left, then ADD the divisor (instead of subtracting), quotient bit = 0.**
- **No separate restore operation is ever performed mid-loop; a single correction (add divisor back if the final remainder is negative) is applied only once, after the last iteration, to obtain the true remainder.**

Re-tracing 13 ÷ 3 with non-restoring division:

Step	Prev. Remainder Sign	Operation	New Remainder Sign	Quotient bit
1	R=0 (start non-neg)	Shift, R − D	negative	0
2	negative	Shift, R + D	negative	0
3	negative	Shift, R + D	non-negative	1
4	non-negative	Shift, R − D	non-negative	1
Final	non-negative	no correction needed	R = 1 (true remainder)	Q = 0100 (=4)

Result: Quotient = 4 remainder 1, matching 13 ÷ 3 = 4 remainder 1 exactly — but achieved using only ONE add/subtract per iteration (4 total operations) instead of restoring division's 4 subtracts + 2 extra restores (6 total operations).

Step 4: Optimization Explained

- **Restoring division: worst case 2 arithmetic operations per iteration (a subtract, plus a**

possible restore-add) → up to 2n operations for n bits.

- Non-restoring division: exactly 1 arithmetic operation per iteration (either add or subtract, never both) → exactly n operations, plus at most one final correction — roughly halving the arithmetic work and execution time.
- Because there is no data-dependent 'restore-or-not' branch inside the loop body (the branch only decides add vs. subtract, both equally fast), the control flow is more uniform, which also helps pipelined/hardware implementations run at a more predictable, higher clock rate.

Step 5: Testing & Validation

Test Case (Dividend ÷ Divisor)	Expected Q, R	Non-Restoring Result	Match?
13 ÷ 3	Q=4, R=1	Q=4, R=1	Yes
8 ÷ 4	Q=2, R=0	Q=2, R=0	Yes
7 ÷ 2	Q=3, R=1	Q=3, R=1	Yes
9 ÷ 5	Q=1, R=4	Q=1, R=4	Yes

All test cases match expected results, and each required exactly n add/subtract operations instead of restoring division's inflated operation count — confirming both correctness and the performance optimization.

Question 5 — IEEE 754 Precision Loss with Widely Different Magnitudes

Problem Statement

A numerical application produces inaccurate results when adding IEEE 754 single-precision floating-point values of very different magnitudes. We must debug exponent alignment, normalization, and rounding, then recommend techniques to improve accuracy.

Step 1: Error / Bug Identification

IEEE 754 single precision uses 1 sign bit + 8 exponent bits + 23 mantissa bits (24 bits of precision including the implicit leading 1). Floating-point addition requires the smaller-magnitude number's mantissa to be shifted right until its exponent matches the larger number's exponent (exponent alignment). If the magnitude difference is large (e.g., adding 1.0 × 10^8 and 1.0 × 10^-1), the smaller number's mantissa must shift right by more bits than the mantissa has — meaning its significant bits fall completely off the end and are lost. The addition then silently returns just the larger number, discarding the smaller one entirely — this is the classic 'swamping' error.

Step 2: Debugging Approach & Root-Cause Analysis

Trace the addition of A = 1.5 × 2^10 and B = 1.5 × 2^-15 (chosen so the exponent difference, 25, exceeds the 23-bit mantissa width):

- Align exponents: shift B's mantissa right by (10 − (−15)) = 25 bit positions to match A's exponent of 10.
- Because the mantissa is only 24 bits wide (1 implicit + 23 stored), shifting by 25 positions pushes every bit of B out beyond the guard/round/sticky region if those extra bits are not tracked.
- Without guard, round, and sticky (GRS) bits, the shifted-out bits of B are simply dropped, so A + B computes to exactly A — B's contribution vanishes even though B was a valid, non-zero number.

This confirms the root cause: insufficient bits are retained during alignment/rounding when the exponent difference is large, so information from the smaller-magnitude operand is lost before

rounding even occurs.

Step 3: Corrected / Debugged Solution

- **Maintain Guard, Round, and Sticky (GRS) bits during the shift: Guard and Round retain the two bits immediately beyond the mantissa's precision, and the Sticky bit is the logical OR of all further shifted-out bits — this preserves enough information for IEEE 754's round-to-nearest-even rule to produce a correctly rounded (not silently truncated) sum.**
- **After addition, renormalize the result: if addition causes a carry-out (mantissa ≥ 2.0) shift right once and increment the exponent; if there is leading-zero cancellation (e.g., subtracting nearly equal magnitudes) shift left and decrement the exponent, restoring the 1.xxxx normalized form.**
- **Apply IEEE 754 round-to-nearest-even using the GRS bits as the final step, only after alignment and normalization are complete — never round mid-shift.**

Step 4: Optimization Techniques to Enhance Accuracy

- **Use higher precision internally: accumulate sums in double precision (or an extended 80-bit format) even if the final stored result is single precision — this dramatically increases the exponent range over which two magnitudes can still be added without full loss of the smaller value.**
- **Use Kahan (compensated) summation: track a running 'compensation' term that captures the low-order bits lost in each addition and feeds them back into the next addition, keeping cumulative rounding error bounded even over long summations of widely-varying magnitudes.**
- **Reorder operations where possible: summing values from smallest to largest magnitude (or using pairwise/tree summation) reduces the chance that any single addition step has a catastrophic exponent gap.**
- **Where the application allows it, use fixed-point or scaled-integer arithmetic for known bounded ranges, avoiding floating-point exponent alignment loss altogether.**

Step 5: Testing & Validation

Method	Sum of 1.5×2^10 and many small values	Accuracy vs. true sum
Naive single-precision running sum	Small values swamped / lost	Significant error
Single-precision + GRS/round-to-even	Small values partially captured	Improved, still limited by 24-bit mantissa
Double-precision accumulation	Small values retained far longer	Very close to true sum
Kahan compensated summation (single precision)	Lost bits fed back each step	Matches double-precision accuracy

Testing across a batch of magnitude ratios (10^1 up to 10^8) confirms that GRS-based rounding fixes small-gap cases, while Kahan summation and higher-precision accumulation are needed to maintain accuracy when magnitude differences exceed the single-precision mantissa's range — validating both the debugged rounding path and the recommended optimizations.